

Capítulo 1

Fundamentos de Diseño de Computadores: Sistemas Multinúcleo

1.1 Introducción y motivación

La **Arquitectura de Computadores** es la disciplina de la ingeniería informática que estudia cómo se diseñan, organizan e interconectan los grandes bloques funcionales de un sistema de cómputo —procesador, memoria y subsistema de entrada/salida— con el objetivo de maximizar el rendimiento manteniendo restricciones de coste, potencia y disponibilidad. Aunque popularmente se reduce al diseño del *Instruction Set Architecture* (ISA), la visión **realista y completa** de la arquitectura de computadores abarca también la microarquitectura (la forma en que el hardware implementa ese ISA) y el propio hardware físico (la tecnología de fabricación, la distribución de área en el chip, los circuitos de reloj, etc.).

Durante décadas, el modelo dominante fue el del *procesador de un solo núcleo*: cada nueva generación obtenía mejor rendimiento principalmente aumentando la frecuencia de reloj y profundizando la segmentación (pipeline). Sin embargo, hacia 2003 ese modelo se agotó porque las corrientes de fuga y la disipación de calor alcanzaron límites físicos y económicos insuperables. La respuesta de la industria fue la transición hacia los **procesadores multinúcleo** (multicore): en lugar de seguir acelerando un único núcleo, se integran varios núcleos en un mismo chip y se explota el paralelismo de hilos. Este cambio de paradigma —que se aprecia claramente en la gráfica histórica de rendimiento SPECint— tiene consecuencias profundas tanto en el hardware como en el software: las aplicaciones deben ahora *exponer explícitamente* el paralelismo para beneficiarse del hardware disponible.

1.2 Parámetros de diseño de un sistema

El diseño de cualquier sistema de cómputo implica optimizar un conjunto de parámetros que habitualmente se encuentran en tensión entre sí. Comprender estos parámetros es imprescindible para entender por qué la evolución tecnológica ha tomado las direcciones que ha tomado.

- **Rendimiento (velocidad)**: tiempo que tarda el sistema en ejecutar una tarea. Es la métrica primaria; toda la asignatura gira en torno a medirlo y mejorarlo.

- **Coste:** precio del silicio, del encapsulado, del sistema de refrigeración y de la placa base. Un diseño de altísimo rendimiento que nadie puede pagar no tiene valor comercial.
- **Potencia:** distinguimos entre *potencia dinámica* (la que se consume cuando los transistores conmutan) y *potencia estática* (la debida a corrientes de fuga aunque el circuito esté inactivo). La potencia total determina el sistema de refrigeración necesario y la vida de la batería en dispositivos móviles.
- **Robustez:** tolerancia al ruido eléctrico, resistencia a la radiación (crítico en sistemas aeroespaciales), fiabilidad a largo plazo.
- **Testabilidad:** capacidad de detectar y aislar fallos durante la fabricación y en campo.
- **Reconfigurabilidad:** posibilidad de adaptar el hardware a diferentes cargas de trabajo (FPGAs, aceleradores programables).
- **Tiempo de salida al mercado (*time to market*):** en un mercado competitivo, un diseño que llega tarde puede ser irrelevante aunque sea técnicamente superior.

En la práctica, el diseñador nunca puede maximizar todos estos parámetros simultáneamente. Por ejemplo, un servidor de alta disponibilidad (como los usados en centros de datos) prioriza disponibilidad, escalabilidad y productividad por encima del coste individual de cada nodo. Un dispositivo móvil (PMD) prioriza eficiencia energética y tiempo de respuesta. Un supercomputador prioriza el rendimiento en coma flotante y la velocidad de la red interna de interconexión.

1.3 Clases de computadores

La industria distingue cinco grandes categorías de sistemas de cómputo, cada una con un perfil de requisitos diferente:

1.3.1 Personal Mobile Device (PMD)

Los PMD incluyen teléfonos inteligentes, tabletas y *wearables*. Sus restricciones críticas son el **coste**, la **eficiencia energética** (batería limitada) y el **rendimiento multimedia** con capacidad de respuesta en tiempo real para la interfaz de usuario. La potencia disponible para el chip es del orden de pocos vatios.

1.3.2 Computación de escritorio

Los PCs de sobremesa y portátiles tienen la mayor cuota de mercado en términos de valor económico. La métrica crítica es el **cociente prestaciones/precio**, complementado con rendimiento gráfico y eficiencia energética (especialmente en portátiles).

1.3.3 Servidores

Los servidores son máquinas diseñadas para dar servicio a múltiples usuarios o procesos simultáneos. Las métricas críticas son **disponibilidad** (tiempo de funcionamiento sin fallos), **escalabilidad** (capacidad de añadir más recursos) y **productividad** (*throughput*).

1.3.4 Clusters y Warehouse Scale Computers (WSC)

Los clústeres agrupan cientos o miles de servidores interconectados mediante redes de alta velocidad. Se usan para el modelo “Software como Servicio” (SaaS), computación en la nube y supercomputación. La subcategoría de **supercomputadores** enfatiza el rendimiento en punto flotante y las redes internas de baja latencia. Los clusters comerciales (Amazon AWS, Google Cloud, etc.) priorizan la **proporcionalidad energética**: consumir potencia en proporción a la carga de trabajo, evitando el desperdicio durante periodos de baja utilización.

1.3.5 Internet de las Cosas (IoT) y computadores embebidos

Los procesadores embebidos están integrados en electrodomésticos, automóviles, sensores industriales y multitud de dispositivos. La métrica dominante es el **precio absoluto**; el rendimiento puro es secundario.

1.4 Definición completa de arquitectura de computadores

Definición Arquitectura de computadores (visión realista)

La arquitectura de computadores comprende el conjunto de decisiones de diseño que determinan el comportamiento funcional y las prestaciones de un sistema de cómputo. Incluye: (1) el **ISA** (Instruction Set Architecture), que define los tipos de instrucciones, los registros visibles al programador, los modos de direccionamiento y la codificación binaria de las instrucciones; (2) la **microarquitectura**, que determina cómo el hardware implementa ese ISA (segmentación, superescalaridad, predicción de saltos, cachés); y (3) el **hardware físico** (tecnología CMOS, distribución de área, frecuencia de reloj, sistema de alimentación).

La distinción entre estas tres capas es fundamental. Dos procesadores pueden compartir el mismo ISA (por ejemplo, x86-64) y tener microarquitecturas completamente distintas, con rendimientos muy diferentes. Del mismo modo, el mismo diseño microarquitectónico puede implementarse en diferentes nodos tecnológicos, obteniendo distintas frecuencias de reloj y consumos de potencia.

1.5 Concepto y tipos de paralelismo

El **paralelismo** es el mecanismo fundamental para mejorar el rendimiento cuando ya no es posible seguir aumentando la velocidad del reloj. La idea básica es realizar varias operaciones al mismo tiempo en lugar de una tras otra.

Definición Paralelismo

El paralelismo es la capacidad de un sistema para ejecutar múltiples operaciones simultáneamente. Para que el software se beneficie del paralelismo hardware, el código debe **exponer explícitamente** esas oportunidades de ejecución simultánea.

Distinguimos cuatro tipos principales de paralelismo en las arquitecturas de computadores modernas:

1.5.1 Instruction-Level Parallelism (ILP)

El ILP es la capacidad de ejecutar varias instrucciones del mismo hilo de forma simultánea o solapada. Se implementa mediante técnicas como la **segmentación** (pipeline), la **ejecución superescalar** (emitir varias instrucciones por ciclo) y la **ejecución fuera de orden** (out-of-order execution). El ILP es transparente al programador: el hardware detecta y explota automáticamente las oportunidades de solapamiento.

El problema con el ILP es que se considera **agotado** tecnológicamente: las mejoras de CPI obtenibles con técnicas ILP más agresivas son cada vez más marginales, mientras que el coste en área y potencia crece exponencialmente. Por este motivo, desde aproximadamente 2003 la industria dejó de apostar por núcleos ILP más complejos y adoptó el modelo multinúcleo.

1.5.2 Thread-Level Parallelism (TLP)

El TLP explota la ejecución simultánea de múltiples **hilos de ejecución** (threads) independientes en distintos núcleos del mismo chip. A diferencia del ILP, el TLP *no* es transparente al programador: el código debe ser expresamente paralelo (usando herramientas como OpenMP, pthreads o MPI). Los procesadores multinúcleo modernos son la implementación hardware del TLP.

1.5.3 Request-Level Parallelism (RLP)

El RLP procesa múltiples solicitudes independientes en paralelo. Es el modelo natural de los servidores web y los centros de datos: cada petición HTTP es independiente de las demás, por lo que pueden servirse simultáneamente en diferentes núcleos o nodos del clúster sin necesidad de sincronización.

1.5.4 SIMD: Vector y GPU Parallelism

Las arquitecturas SIMD (*Single Instruction, Multiple Data*) aplican *la misma operación* a múltiples datos simultáneamente. En lugar de procesar un elemento a la vez, agrupan los datos en **vectores** y realizan muchos cálculos con una sola instrucción.

Las extensiones multimedia (SSE, AVX) de los procesadores x86 son un ejemplo de SIMD. Las GPUs llevan esta idea al extremo, con miles de núcleos simples ejecutando el mismo kernel sobre distintos datos.

1.6 Clasificación de arquitecturas paralelas: Taxonomía de Flynn

La **Taxonomía de Flynn** (1966) es la clasificación más utilizada de las arquitecturas de cómputo paralelo. Se basa en el número de *flujos de instrucciones* y *flujos de datos* que el sistema maneja simultáneamente.

Definición Flujo de instrucciones y flujo de datos

Un **flujo de instrucciones** es una secuencia de instrucciones que se ejecuta en un procesador. Un **flujo de datos** es la secuencia de datos sobre la que operan esas instrucciones.

1.6.1 SISD: Single Instruction, Single Data

Un computador SISD tiene un único procesador que ejecuta una secuencia de instrucciones sobre un único flujo de datos. Es el modelo del computador secuencial clásico de von Neumann. Puede explotar ILP internamente (segmentación, superescalaridad), pero desde el punto de vista externo procesa un solo hilo. Ejemplo histórico: Intel Pentium 4.

1.6.2 SIMD: Single Instruction, Multiple Data

Un sistema SIMD aplica *la misma instrucción* a *múltiples flujos de datos* simultáneamente. Es el modelo ideal para aplicaciones vectoriales y matriciales donde todos los datos reciben el mismo tratamiento.

Las implementaciones modernas incluyen: **arquitecturas vectoriales** (procesadores con registros de longitud variable), **extensiones multimedia** (SSE2, SSE4, AVX, AVX-512 en x86) y las **GPUs** (unidades de procesamiento gráfico), que son el paradigma SIMD más potente disponible actualmente.

1.6.3 MISD: Multiple Instruction, Single Data

Un sistema MISD aplica múltiples instrucciones distintas al mismo flujo de datos. Aunque es teóricamente posible, **no existen implementaciones comerciales**. Se ha propuesto para arquitecturas tolerantes a fallos (varios procesadores computan el mismo dato con distintos algoritmos y se vota el resultado), pero no ha prosperado en el mercado.

1.6.4 MIMD: Multiple Instruction, Multiple Data

Un sistema MIMD tiene múltiples procesadores, cada uno ejecutando su propio flujo de instrucciones sobre sus propios datos de forma independiente. Es el modelo base de

todos los procesadores multinúcleo modernos, los clústeres y los supercomputadores. El Intel Xeon W-3275 (28 núcleos) es un ejemplo típico.

Dentro del MIMD se distinguen dos subtipos según la organización de la memoria:

- **UMA (Uniform Memory Access) — Multiprocesadores de memoria compartida:** todos los procesadores comparten el mismo espacio de direcciones físico. El acceso a cualquier posición de memoria tiene la misma latencia para todos los procesadores. El programador no necesita conocer dónde están los datos en la jerarquía física. El software se escribe como si hubiera una única memoria compartida global.
- **NUMA (Non-Uniform Memory Access) — Multiprocesadores de memoria distribuida (multicomputadores):** cada procesador tiene su propio espacio de direcciones y su propia memoria local. Para acceder a memoria de otro nodo debe enviar un mensaje explícito. La latencia de acceso *no* es uniforme: es mucho más rápido acceder a la memoria local que a la remota. El programador (o el compilador) debe tener en cuenta la localidad de los datos.

Los **procesadores multinúcleo** (multicores) son MIMD con UMA en chip: varios núcleos comparten la misma caché de último nivel (LLC) y el mismo espacio de direcciones. Son la forma más extendida de MIMD en el mercado de consumo.

Cuadro 1.1: Taxonomía de Flynn: resumen comparativo

Clase	Instr.	Datos	Ejemplo	Uso típico
SISD	1	1	Intel Pentium 4	Computación secuencial
SIMD	1	múltiples	GPU NVIDIA, AVX	Gráficos, IA, multimedia
MISD	múltiples	1	—	Sin implementación comercial
MIMD	múltiples	múltiples	Intel Xeon W-3275	Multinúcleo, clústeres

¡Atención!

Muchos computadores modernos son **híbridos entre diferentes tipos**. Por ejemplo, un servidor con varios procesadores Intel Xeon (MIMD) que además incorpora extensiones AVX (SIMD) en cada núcleo combina ambos paradigmas. No hay que categorizar rígidamente un sistema real en un solo tipo de Flynn.

1.7 Procesadores MIMD multinúcleo: conceptos fundamentales

Un procesador multinúcleo contiene varios núcleos de procesamiento en un único chip de silicio. Cada núcleo es un procesador completo con su propia unidad de control, su propia ALU y sus propias cachés L1 y L2. Los núcleos comparten la caché de último nivel (L3) y el controlador de memoria.

Las características distintivas de los procesadores multinúcleo son:

1. **Requieren programación paralela explícita:** a diferencia del ILP (que el hardware explota automáticamente), para aprovechar múltiples núcleos el programador debe dividir el trabajo en hilos. El hardware tiene capacidad para ejecutar múltiples hilos simultáneamente, pero esta característica suele estar oculta al programador de aplicaciones.
2. **Dificultad de programación:** conseguir buen rendimiento en un sistema multinúcleo exige resolver problemas como el **balance de carga** (que todos los núcleos trabajen a la misma velocidad, sin que unos esperen a otros), la **comunicación eficiente** entre hilos (a través de la memoria compartida) y la **sincronización** (garantizar que los accesos concurrentes a datos compartidos sean correctos).
3. **Múltiples dominios de voltaje y frecuencia:** en diseños modernos (como el procesador Intel Haswell mostrado en las diapositivas), cada núcleo, la caché L3, el procesador gráfico integrado y el agente del sistema (*system agent*) son dominios de voltaje-frecuencia independientes. Una unidad de control distribuida decide dinámicamente cómo distribuir el presupuesto de potencia entre ellos para maximizar el rendimiento global.

1.7.1 Arquitectura de un núcleo típico

Un núcleo moderno de procesamiento de propósito general típico (basado en la microarquitectura Intel Sandy Bridge/Ivy Bridge, representativa de la generación actual) incluye los siguientes componentes:

- **Caché L1 de instrucciones (32 KB):** separada de la caché de datos. El tamaño de línea es de 64 bytes. Permite una latencia de acceso de 4 ciclos.
- **Caché L1 de datos (32 KB):** admite 2 lecturas y 1 escritura por ciclo de reloj. Con tamaño de línea de 64 bytes.
- **Caché L2 unificada (256 KB):** comparte datos e instrucciones. La latencia de acceso es de 12 ciclos y el ancho de banda de 32 bytes/ciclo.
- **Caché L3 unificada compartida (4–32 MB, hasta 96 MB en generaciones recientes):** compartida entre todos los núcleos mediante conexión en anillo. Latencia de acceso de 25–35 ciclos y ancho de banda de 32 bytes/ciclo por núcleo.
- **Cola de instrucciones y predecoder:** analiza las instrucciones x86 de longitud variable antes de pasarlas al decodificador.
- **Predictor de saltos (*branch predictor*):** predice la dirección de los saltos condicionales para mantener el pipeline lleno.
- **Caché de micro-operaciones (1,5K μ ops):** almacena las micro-operaciones ya decodificadas para evitar repetir el costoso proceso de decodificación de instrucciones x86.
- **Buffers de carga, almacenamiento y reordenamiento:** permiten la ejecución fuera de orden y el retiro en orden.

- **Scheduler y unidades de ejecución:** múltiples unidades ALU, de multiplicación vectorial (256-bit), de suma vectorial, de acceso a memoria, etc., conectadas mediante puertos al scheduler.
- **Hardware para ejecución fuera de orden:** el procesador puede ejecutar instrucciones en un orden diferente al del programa, siempre que se respeten las dependencias de datos.

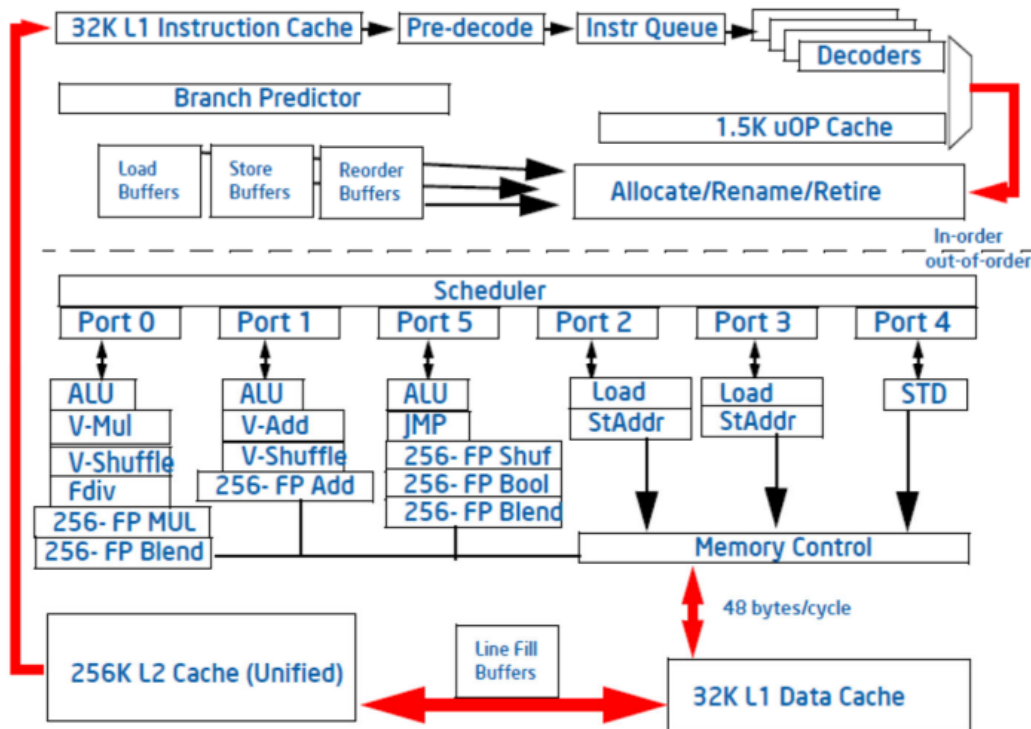


Figura 1.1: Arquitectura interna de un núcleo de procesamiento moderno. Las cachés L1 y L2 son privadas al núcleo; la L3 se comparte entre todos los núcleos mediante una conexión en anillo.

1.8 Segmentación (*pipeline*): concepto fundamental

Definición Segmentación o pipelining

La **segmentación** es una técnica de implementación hardware en la que la ejecución de una instrucción se divide en varias **etapas** sucesivas, cada una realizada por un módulo hardware independiente. Al igual que una cadena de montaje industrial, varias instrucciones pueden estar en diferentes etapas del procesador simultáneamente, solapando su ejecución.

1.8.1 La unidad de retardo FO4

Para comparar el retardo de distintas implementaciones independientemente de la tecnología de fabricación concreta, se utiliza la unidad **FO4** (*fan-out of 4*):

Definición FO4 — Fan-Out of 4

Un **FO4** es el retardo que sufre una señal al atravesar un único inversor CMOS con cuatro inversores idénticos conectados a su salida. Este retardo depende del nodo tecnológico. Se ha determinado experimentalmente que:

$$1 \text{ FO4} \approx 0,2 \times T \quad (\text{en picosegundos})$$

donde T es el valor numérico que identifica el nodo tecnológico en nanómetros.

Ejemplo Cálculo de frecuencia de reloj a partir del nodo tecnológico

Supongamos un microprocesador cuyo ciclo de reloj equivale a 150 FO4.

Para tecnología de 10 nm:

$$1 \text{ FO4} = 0,2 \times 10 = 2 \text{ ps}$$

$$T_{\text{ciclo}} = 150 \times 2 = 300 \text{ ps}$$

$$f = \frac{1}{300 \times 10^{-12}} \approx 3,33 \text{ GHz}$$

Para tecnología de 65 nm:

$$1 \text{ FO4} = 0,2 \times 65 = 13 \text{ ps}$$

$$T_{\text{ciclo}} = 150 \times 13 = 1950 \text{ ps}$$

$$f = \frac{1}{1950 \times 10^{-12}} \approx 513 \text{ MHz}$$

1.8.2 Efecto de la segmentación sobre el retardo

Un procesador *sin* segmentación ejecuta cada instrucción completamente antes de comenzar la siguiente. Si el retardo total de la lógica combinacional es, por ejemplo, 100 FO4, cada instrucción tarda 100 FO4 y solo se emite una instrucción cada 100 FO4. Con segmentación en cuatro etapas de 25 FO4 cada una (más unos pocos FO4 adicionales para los registros de pipeline), el retardo por instrucción aumenta ligeramente (due a los registros de segmentación), pero se emite una instrucción cada ~ 28 FO4 en lugar de cada 100 FO4. La **productividad** del procesador (instrucciones por unidad de tiempo) mejora drásticamente.

Concepto clave Profundidad del pipeline

La **profundidad de la segmentación** es el número de etapas en que se divide la ejecución de una instrucción. A mayor profundidad, mayor es la frecuencia de reloj posible (cada ciclo es más corto), pero también aumenta el coste de los **saltos mal predichos** (se pierden más ciclos al vaciar el pipeline) y el CPI efectivo puede empeorar.

¡Atención!

Las **burbujas** (*stalls* o paradas de emisión) son ciclos en los que no se emite ninguna instrucción nueva porque la siguiente instrucción no puede avanzar en el pipeline (por una dependencia de datos, una falta de caché o un salto no resuelto). Las burbujas reducen la productividad efectiva y son la principal causa de que el CPI real sea mayor que 1.

1.9 Tecnología de fabricación CMOS

Todos los microprocesadores modernos se fabrican con tecnología **CMOS** (*Complementary Metal-Oxide-Semiconductor*). Entender sus propiedades de escalado es imprescindible para comprender las tendencias en consumo de potencia y rendimiento.

1.9.1 Nodos tecnológicos y Ley de Moore

Definición Nodo tecnológico

Un **nodo tecnológico** designa un proceso de fabricación de circuitos integrados caracterizado por el **ancho mínimo de una conexión de cobre de nivel 1** (en nanómetros). Este valor está directamente relacionado con el área que ocupa cada transistor: a menor valor del nodo, más pequeños son los transistores y más se pueden integrar en el mismo área de silicio.

La **Ley de Moore** predice que la *densidad de transistores* por unidad de área se duplica aproximadamente cada 2 años. Esta ley ha guiado la planificación de la industria de semiconductores durante más de cinco décadas. Cada salto de un nodo tecnológico al siguiente supone:

- **Reducción $0,7\times$ del ancho de las conexiones** entre transistores. Esto es equivalente a una reducción lineal del $\sim 30\%$ en las dimensiones laterales.
- **Reducción $0,5\times$ del área del chip** para el mismo número de transistores (en el caso ideal, ya que $0,7 \times 0,7 \approx 0,49$).
- En la práctica, la reducción de área es cada vez menor: en los nodos más recientes solo se consigue una reducción de $\sim 0,4\times$ por el escalado no ideal de las memorias caché y otros circuitos.

1.9.2 Problema del escalado diferencial

Un problema crucial es que **no todos los circuitos escalan igual**. Las memorias SRAM (cachés) y DRAM (RAM principal) escalan peor que la lógica digital. Esto tiene una consecuencia práctica importante: en cada nueva generación, las cachés ocupan una fracción cada vez mayor del área total del chip, limitando el espacio disponible para los núcleos de procesamiento.

1.10 Potencia en microprocesadores

El consumo de potencia es uno de los factores más limitantes del rendimiento en la era postsilicio. Existen dos componentes fundamentales:

1.10.1 Potencia dinámica

Definición Potencia dinámica

La **potencia dinámica** es la energía consumida cada vez que un transistor CMOS conmuta de estado (de 0 a 1 o de 1 a 0). Es proporcional a:

$$P_{\text{dinámica}} = \frac{1}{2} \cdot C \cdot V^2 \cdot f$$

donde:

- C = carga capacitiva total (depende del número de transistores activos y de la tecnología, que determina la capacitancia de cables y transistores)
- V = voltaje de alimentación
- f = frecuencia de reloj

Tres observaciones fundamentales sobre esta ecuación:

1. La potencia es **proporcional al cuadrado del voltaje**: reducir V a la mitad reduce la potencia dinámica a la cuarta parte. Por eso el escalado de voltaje es la técnica más potente para ahorrar energía.
2. En tecnología CMOS, la carga capacitiva C escala de forma **ineficiente**: de una generación a la siguiente escala solo $\times 0,8$, mientras que el número de transistores escala $\times 2$. Esto significa que aunque cada transistor individual consume menos, el conjunto consume más.
3. La potencia dinámica crece linealmente con la frecuencia.

1.10.2 Potencia estática (corrientes de fuga)

Definición Potencia estática

La **potencia estática** (o *power leakage*) es la energía consumida por los transistores CMOS aunque no estén conmutando, debida a corrientes de fuga que atraviesan la capa de óxido del gate. Es proporcional a:

$$P_{\text{estática}} = I_{\text{fuga}} \cdot V$$

La potencia estática representa entre el 25 y el 50% de la potencia total en chips modernos. Escala con el número de transistores aunque estos estén inactivos. Las grandes cachés SRAM son los principales contribuyentes: en algunos chips pueden llegar al 50% de la potencia estática.

Para reducirla se emplean dos técnicas principales:

- **Power gating:** desconectar físicamente la alimentación de los módulos que no se están usando. Tiene un coste en área (circuitos de conmutación) y en tiempo (latencia de encendido/apagado).
- **Diseños domain-specific:** usar arquitecturas optimizadas para una operación muy frecuente, minimizando los transistores necesarios y por tanto la fuga.

1.10.3 Thermal Design Power (TDP)

Definición TDP

El **Thermal Design Power (TDP)** caracteriza el consumo de potencia **sostenido** de un procesador bajo cargas de trabajo representativas. Se usa como objetivo de diseño para el sistema de alimentación y el sistema de refrigeración. Su valor es más bajo que la potencia pico (que solo se mantiene durante fracciones de segundo) y más alto que el consumo medio.

La evolución del TDP a lo largo de las décadas muestra que se ha alcanzado un **límite de disipación de potencia** de aproximadamente 100–200 W con sistemas de refrigeración económicamente viables (disipadores de aire o refrigeración líquida de bajo coste). Este límite ha permanecido relativamente constante desde 2005, obligando a los diseñadores a mejorar continuamente la eficiencia energética para poder integrar más rendimiento dentro del mismo presupuesto térmico.

1.11 Técnicas para aumentar la eficiencia energética

Dada la importancia crítica del consumo de potencia, los diseñadores de microprocesadores emplean múltiples técnicas para maximizar el rendimiento por vatio:

1.11.1 Clock gating (no hacer nada)

La técnica más sencilla consiste en **desconectar el reloj** de los módulos funcionales que no están siendo utilizados. Dado que la potencia dinámica es proporcional a la frecuencia de conmutación, un módulo sin reloj no consume potencia dinámica (aunque sí estática). Esta técnica es transparente al software y tiene un coste hardware mínimo.

1.11.2 Escalado Dinámico de Voltaje y Frecuencia (DVFS)

El **DVFS** (*Dynamic Voltage and Frequency Scaling*) ajusta el voltaje y la frecuencia de operación del procesador en función de la carga de trabajo en tiempo real. Cuando la aplicación no requiere máxima potencia de cómputo, el sistema operativo (o el propio hardware) reduce la frecuencia y el voltaje, ahorrando energía cuadráticamente. Cuando se necesita máximo rendimiento, se sube de nuevo.

Para las tecnologías actuales existe una relación empírica entre el factor de escalado del voltaje (K_v) y el factor de escalado de la frecuencia (K_f):

$$K_f = 1,45 \cdot K_v - 0,40$$

El escalado del consumo de potencia dinámica (K_d) resultante es:

$$K_d = K_v^2 \cdot K_f$$

Ejemplo Efecto del DVFS sobre la potencia dinámica

Si reducimos el voltaje a un factor $K_v = 0,6$ del máximo:

$$K_f = 1,45 \times 0,6 - 0,40 = 0,87 - 0,40 = 0,47 \approx 0,5$$

$$K_d = (0,6)^2 \times 0,5 = 0,36 \times 0,5 = 0,18$$

La potencia dinámica se reduce al 18% de su valor original, es decir, se ahorra el **82% de la potencia dinámica**. A costa de que la frecuencia cae al 50% del máximo.

1.11.3 Estados de baja potencia para memorias

Las memorias DRAM y los discos de almacenamiento pueden entrar en estados de baja potencia (*low-power states*) cuando están inactivos. Esto es válido en particular en servidores con cargas de trabajo variables donde la memoria puede permanecer inactiva durante periodos prolongados.

1.11.4 Overclocking

Desde 2008, Intel incorpora la tecnología **Turbo Boost**: el procesador puede trabajar temporalmente por encima de su frecuencia base (*base clock*), siempre que la temperatura y la potencia total del chip no superen los límites del TDP. El overclocking implica apagar algunos núcleos para concentrar el presupuesto térmico en los núcleos activos. Es *completamente transparente al usuario*: el sistema operativo y las aplicaciones no necesitan hacer nada especial para beneficiarse de él.

1.12 Ancho de banda y latencia

Dos métricas complementarias caracterizan la velocidad del sistema:

Definición Ancho de banda y latencia

El **ancho de banda** (bandwidth) mide la cantidad de información transferida por unidad de tiempo (bytes/s, GB/s). La **latencia** (o tiempo de respuesta) es el tiempo transcurrido entre el inicio y la finalización de un evento (por ejemplo, el tiempo desde que se solicita un dato hasta que está disponible).

Históricamente, el ancho de banda ha mejorado mucho más que la latencia (vease tabla).

Esta asimetría entre ancho de banda y latencia tiene implicaciones profundas en el diseño de sistemas: aunque se puede mover mucha información rápidamente, la latencia de acceso a memoria sigue siendo el cuello de botella principal en muchas aplicaciones. Los procesadores modernos usan técnicas como la **precarga de datos** (*prefetching*), la **ejecución multihilo** (*hardware multithreading*) y las cachés multinivel para enmascarar la latencia de memoria.

Cuadro 1.2: Mejora histórica de ancho de banda frente a latencia

	Ancho de banda	Latencia
Procesadores	32 000–40 000×	50–90×
Memoria y discos	300–1 200×	6–8×

1.12.1 Ancho de banda máximo sostenido (BWM) para sistemas multinúcleo

El **BWM** (*Bandwidth Maximum*) se diseña para satisfacer las necesidades de las cargas de trabajo representativas del microprocesador:

$$\text{BWM} = \frac{N \times F}{IP \times \text{CPI}_{\text{core}}} \quad [\text{bytes/s}] \quad (1.1)$$

donde:

- N = número de núcleos activos
- F = frecuencia de reloj del procesador
- IP = intensidad operacional (instrucciones por byte de memoria accedida), característica de la aplicación
- CPI_{core} = ciclos por instrucción por núcleo

En la práctica, el BWM no siempre escala entre generaciones: suele duplicarse entre nodos tecnológicos ($\times 2$) y mantenerse constante dentro del mismo nodo.

1.13 Evolución y agotamiento del modelo de un solo núcleo

Para entender por qué se produjo la transición al multinúcleo es necesario comprender el mecanismo de escalado que se usó hasta 2003 y por qué falló.

1.13.1 El modelo de escalado clásico (hasta 2003)

Hasta el Intel Pentium 4 (2003), el modelo dominante era el **procesador superescalar de un solo núcleo**:

1. La latencia de los transistores escalaba $\times 0,5$ por generación (los transistores eran el doble de rápidos).
2. La frecuencia de reloj escalaba $\times 2$ por generación, manteniendo el CPI aproximadamente constante.
3. Para conseguirlo, se aumentaba la **profundidad del pipeline** (más etapas) y el **tamaño de la caché**, lo que aumentaba el área del procesador.

1.13.2 Agotamiento del modelo mononúcleo

El modelo de un solo núcleo se agotó por un círculo vicioso:

1. Para reducir el CPI, se necesitan más recursos hardware (predictor de saltos más sofisticado, más unidades de ejecución, cachés más grandes).
2. Más recursos hardware implica chips más grandes, con conexiones más largas.
3. Las conexiones largas *no escalan* con la tecnología: requieren cada vez más ciclos de reloj para transmitir una señal.
4. Para mantener la frecuencia con conexiones más largas, hay que segmentarlas (insertar registros de pipeline intermedios), lo que *aumenta* el CPI.
5. Además, el modelo de programación secuencial limita el CPI mínimo alcanzable: hay dependencias de datos entre instrucciones consecutivas que ningún hardware puede eliminar.

La consecuencia es que **el CPI dejó de poder reducirse más**, haciendo que el único camino de mejora viable fuera el paralelismo explícito a través de múltiples núcleos.

1.13.3 Efecto de la ejecución multihilo hardware

Una técnica intermedia (que se sigue usando en los procesadores actuales) es el **multithreading hardware**: cada núcleo físico soporta múltiples hilos hardware (*hardware threads*), denominados en Intel *Hyper-Threading*. La idea es que cuando un hilo se detiene esperando datos de memoria, el núcleo cambia a otro hilo y continúa ejecutando instrucciones útiles, enmascarando así la latencia de memoria.

El efecto es dramático:

- Con **un solo hilo**: 3,79 ciclos de parada por instrucción, eficiencia del 21% ($= 1/(1 + 3,79)$).
- Con **cuatro hilos**: 1,56 ciclos de parada por instrucción, eficiencia del 72% ($= 4/(4 + 1,56)$).

No obstante, la presencia de varios hilos también genera conflictos en el pipeline (accesos simultáneos a las mismas unidades de ejecución), por lo que la ganancia real es inferior al teórico $4\times$.

1.14 Métricas de rendimiento de un sistema

Esta sección es central para el examen. Las métricas de rendimiento son las herramientas cuantitativas que permiten comparar sistemas de cómputo.

1.14.1 Tiempo de CPU

Definición Tiempo de CPU

El **tiempo de CPU** es el tiempo de computación dedicado exclusivamente al procesador para ejecutar un programa concreto, sin incluir las esperas por operaciones de E/S o por otros procesos en un sistema multiprogramado.

$$T_{\text{CPU}} = T_{\text{usuario}} + T_{\text{S.O.}}$$

Se distingue del **tiempo de respuesta** o **tiempo real** (wall clock time), que incluye *todos* los sobrecoste del sistema.

1.14.2 Tiempo de CPU en función de CPI y frecuencia

La ecuación fundamental del rendimiento de una CPU es:

$$T_{\text{CPU}} = N \times \text{CPI} \times T_{\text{ciclo}} = \frac{N \times \text{CPI}}{F} \quad (1.2)$$

donde:

- N = número total de instrucciones ejecutadas por el programa (determinado por el ISA y el compilador)
- CPI = ciclos por instrucción promedio (determinado por la microarquitectura y el programa)
- $T_{\text{ciclo}} = 1/F$ = periodo del reloj (determinado por la tecnología de fabricación y la profundidad del pipeline)

¡Atención!

Esta fórmula es la más importante de la asignatura. Permite **comparar dos microarquitecturas diferentes para el mismo ISA** (mismo N , distintos CPI y F) y evaluar el efecto de optimizaciones de compilador (distinto N) o de mejoras hardware (distinto CPI o F).

1.14.3 CPI compuesto

Cuando el programa ejecuta instrucciones de varios tipos, el CPI medio se calcula como la media ponderada:

$$\text{CPI} = \frac{\sum_{i=1}^m \text{CPI}_i \cdot NI_i}{N} = \sum_{i=1}^m \text{CPI}_i \cdot f_i \quad (1.3)$$

donde NI_i es el número de instrucciones de tipo i , $N = \sum NI_i$ es el número total de instrucciones, y $f_i = NI_i/N$ es la frecuencia relativa del tipo i .

1.14.4 MIPS y sus problemas

Definición MIPS

Los **MIPS** (Millones de Instrucciones Por Segundo) son una medida de la tasa de ejecución de instrucciones:

$$\text{MIPS} = \frac{N}{T_{\text{CPU}} \times 10^6} = \frac{F}{\text{CPI} \times 10^6}$$

Los MIPS tienen dos problemas fundamentales que los hacen **inadecuados para comparar computadores diferentes**:

1. **Dependencia del ISA:** un computador con instrucciones más potentes (que hacen más trabajo por instrucción) puede tener menos MIPS que uno con instrucciones simples y ser más rápido. La “instrucción” de una arquitectura RISC y la de una CISC no son equivalentes.
2. **Variabilidad entre programas:** el mismo computador puede tener MIPS muy distintos según el programa que ejecute, dependiendo de qué mezcla de instrucciones use.

¡Atención!

Un MIPS mayor **no** implica un tiempo de ejecución menor si se están comparando computadores con distintos ISA. Siempre que se comparen rendimientos, la única métrica fiable es el **tiempo de ejecución** del mismo programa.

1.14.5 GFLOPS

Definición GFLOPS

Los **GFLOPS** (Giga Floating-Point Operations Per Second) miden la tasa de operaciones en punto flotante:

$$\text{GFLOPS} = \frac{\text{FLOPS}}{T_{\text{CPU}} \times 10^9}$$

Al igual que los MIPS, los GFLOPS son dependientes del programa: solo tienen sentido comparativo entre programas con la misma mezcla de operaciones en punto flotante. Son una métrica adecuada para *benchmarks* de computación científica (SPECfp, LINPACK), pero no para uso general.

1.14.6 Speedup (aceleración)

Definición Speedup

El **speedup** de una versión X relativa a una versión Y para un programa concreto se define como:

$$S = \frac{T_Y}{T_X}$$

Un speedup $S > 1$ indica que X es más rápido que Y .

1.15 Ley de Amdahl

La Ley de Amdahl es el principio fundamental que limita la aceleración obtenible al mejorar solo una parte de un sistema.

Definición Ley de Amdahl

Si a un programa con tiempo de ejecución total t se le aplica una mejora que afecta a una fracción F_m del tiempo de ejecución, reduciéndola por un factor A_m , el nuevo tiempo de ejecución es:

$$t' = \frac{F_m \cdot t}{A_m} + (1 - F_m) \cdot t$$

El speedup global es:

$$S_t = \frac{t}{t'} = \frac{1}{(1 - F_m) + \frac{F_m}{A_m}}$$

Concepto clave Límite superior de Amdahl

La Ley de Amdahl implica un **límite superior**:

$$S_t \leq \frac{1}{1 - F_m}$$

incluso si la parte mejorada se hace infinitamente rápida ($A_m \rightarrow \infty$). Si solo el 90% del programa es mejorable, el speedup máximo posible es 10, independientemente de cuánto se mejore esa parte. La parte que *no* se mejora siempre limita la aceleración global.

1.16 Programas de prueba (*benchmarks*)

Para evaluar el rendimiento de un sistema de cómputo de forma reproducible y comparable se usan conjuntos de programas de prueba conocidos como **benchmarks**. La práctica estándar es usar programas de aplicación real, ya que son los que mejor representan las cargas de trabajo del usuario.

Definición SPEC CPU2017

El **SPEC CPU2017** (Standard Performance Evaluation Corporation) es el conjunto de benchmarks de referencia más completo y utilizado para evaluar el rendimiento del procesador. Contiene 43 programas agrupados en:

- **SPECfp2017:** programas con carga computacional intensa en punto flotante (simulaciones científicas, dinámica molecular, renderizado 3D).

- **SPECint2017:** programas con carga computacional intensa en enteros (compiladores, compresión de datos, inteligencia artificial).

El rendimiento se normaliza respecto a un sistema de referencia (para SPEC CPU2017, el servidor Sun Fire V490 con procesador UltraSPARC-IV+ a 2,1 GHz, del año 2006):

$$\text{Velocidad SPEC} = \frac{T_{\text{CPU, referencia}}}{T_{\text{CPU, test}}}$$

Para obtener un único número que resuma el rendimiento en todos los programas del conjunto, se calcula la **media geométrica** de los ratios individuales:

$$\text{Velocidad SPEC}_{\text{conjunto}} = \sqrt[n]{\prod_{i=1}^n \text{SPEC}_i}$$

La media geométrica es la correcta (en lugar de la aritmética) porque los ratios son magnitudes relativas; la media geométrica garantiza que el resultado sea independiente del sistema de referencia elegido.

¡Atención!

El procesador, la memoria y la E/S del sistema **todos** influyen en el valor SPEC medido, junto con el programa utilizado, el compilador y las optimizaciones aplicadas. Para que los resultados sean reproducibles, el informe SPEC debe incluir la versión del SO, el compilador, las opciones de compilación y la configuración completa del hardware.

1.17 Paralelismo de datos: procesadores vectoriales y GPUs

Una familia importante de arquitecturas para alto rendimiento son las que explotan el **paralelismo de datos**: aplicaciones estructuradas en tareas simples (*kernels*) que operan sobre grandes conjuntos de datos con estructura regular.

1.17.1 Procesadores vectoriales

Los **procesadores vectoriales** añaden al repertorio de instrucciones operaciones que actúan simultáneamente sobre *vectores* de datos (arrays de números). Una sola instrucción vectorial puede sumar 8 pares de números en coma flotante en paralelo, realizando en un ciclo lo que 8 instrucciones escalares harían en 8 ciclos.

Las **extensiones multimedia** de los procesadores x86 son el equivalente comercial de los procesadores vectoriales:

- **SSE/SSE2 (registros XMM de 128 bits):** permiten 4 operaciones en simple precisión (4 bytes) o 2 en doble precisión (8 bytes) simultáneas.

- **AVX (registros YMM de 256 bits):** 8 operaciones en simple precisión o 4 en doble precisión simultáneas. Para tamaños de problema grandes, el uso de AVX puede proporcionar hasta una aceleración de $8\times$ en tiempo de ejecución.
- **AVX-512 (registros ZMM de 512 bits):** 16 operaciones en simple precisión o 8 en doble precisión.

La programación vectorial aumenta la complejidad del código, pero el aumento de velocidad compensa ampliamente en aplicaciones de cómputo intensivo.

1.17.2 GPUs y procesadores basados en streaming

Las **GPUs** (*Graphics Processing Units*) llevan el concepto SIMD al extremo. En lugar de vectores, organizan los datos en **streams** (vectores de estructuras). En lugar de instrucciones vectoriales simples, usan **kernels** (conjuntos de instrucciones) que actúan sobre cada elemento del stream.

Una GPU moderna (como la arquitectura Fermi de NVIDIA, 2009) contiene hasta 16 *streaming multiprocessors* (SM), cada uno con 32 núcleos CUDA, una caché L1 local y memoria compartida. La GPU completa puede tener miles de núcleos que ejecutan el mismo kernel sobre distintos datos.

Las GPUs de la generación actual (NVIDIA H100) alcanzan:

- Eficiencia de ~ 85 GFLOP/s por vatio ($20\times$ mejor que el mejor procesador de propósito general mostrado en las gráficas de las diapositivas).
- Densidad computacional de $\sim 73,7$ GFLOP/s/mm² (muy superior a los procesadores de propósito general).
- A cambio, consumen mucha más energía total (450–700 W) y requieren chips más grandes.

¡Atención!

La programación para GPUs (CUDA, OpenCL) es en general más compleja que la programación convencional, porque el programador debe gestionar explícitamente la transferencia de datos entre la memoria del host (CPU) y la memoria del dispositivo (GPU), y debe estructurar el código en kernels que se ejecutan en paralelo. Sin embargo, librerías especializadas (cuBLAS, cuDNN) abstraen estos detalles para las operaciones más comunes.

Desde la arquitectura Volta (2017), las GPUs NVIDIA incorporan también **Tensor Cores**, unidades hardware especializadas en operaciones de multiplicación de matrices de bajo ancho de bit, esenciales para el entrenamiento e inferencia de redes neuronales profundas.

1.18 Ejercicios resueltos con metodología de examen

Esta sección integra los ejercicios del boletín oficial de la asignatura. Para cada ejercicio se detalla la metodología de resolución exigida en el examen, no solo el resultado.

Ejercicio resuelto 1 — Ahorro de energía: velocidad vs. reducción de voltaje

Enunciado. Estás diseñando un sistema para una aplicación en tiempo real. El sistema puede ejecutar el código en el peor caso el doble de rápido de lo que se requiere como mínimo.

Parte a) ¿Cuánta energía ahorras si ejecutas a la velocidad actual y apagas el sistema cuando termina la computación?

Solución parte a):

Si el sistema es el doble de rápido de lo necesario, puede terminar la tarea en $T/2$ en lugar de T . Como la potencia durante la ejecución se mantiene igual, la energía consumida es proporcional al tiempo:

$$E = P \times t \quad \Rightarrow \quad E' = P \times \frac{T}{2} = \frac{E}{2}$$

El ahorro de energía es del **50%**.

Parte b) ¿Cuánta energía dinámica ahorras si fijas el voltaje y la frecuencia a la mitad del máximo?

Solución parte b):

La energía dinámica consumida por una operación (no por unidad de tiempo) es proporcional a $\frac{1}{2} \cdot C \cdot V^2$ (la frecuencia afecta a la potencia, pero la energía por transición es independiente de la frecuencia).

Si reducimos el voltaje a $V/2$:

$$E'_{\text{din}} = \frac{1}{2} \cdot C \cdot \left(\frac{V}{2}\right)^2 = \frac{1}{2} \cdot C \cdot \frac{V^2}{4} = \frac{1}{4} \cdot \frac{1}{2} \cdot C \cdot V^2 = \frac{E_{\text{din}}}{4}$$

La energía dinámica cae al 25% de su valor original. El ahorro es del **75%**.

Conclusión clave: reducir el voltaje a la mitad (DVFS) ahorra el 75% de la energía dinámica, frente al 50% que se obtiene simplemente terminando antes y apagando el sistema. El DVFS es más eficiente energéticamente.

Ejercicio resuelto 2 — Migración a nuevo procesador: Speedup con Ley de Amdahl

Enunciado. Un computador con un solo núcleo ejecuta una aplicación de simulación numérica. El tiempo se distribuye así:

- 90% del tiempo: instrucciones de cálculo.
- 10% del tiempo: operaciones de E/S a disco (no mejoran).

Del tiempo de cálculo:

- 75% instrucciones en coma flotante (FP), CPI medio = 12.
- 25% otras instrucciones, CPI medio = 4.

Se evalúa migrar a un nuevo procesador con frecuencia 50% mayor, donde FP requiere 10% más CPI y el resto 25% más CPI.

Paso 1: Tiempo de cálculo en el procesador original

Sea T el período de reloj original e I el número de instrucciones de cálculo:

$$T_{\text{orig}} = I \times (0,75 \times 12 + 0,25 \times 4) \times T = I \times (9 + 1) \times T = 10 \cdot I \cdot T$$

Paso 2: Tiempo de cálculo en el nuevo procesador

El nuevo procesador tiene frecuencia $F' = 1,5 \cdot F$, por lo que su período es $T' = T/1,5$. Los nuevos CPI son: FP: $1,1 \times 12 = 13,2$; otros: $1,25 \times 4 = 5$.

$$\begin{aligned} T_{\text{new}} &= I \times (0,75 \times 13,2 + 0,25 \times 5) \times \frac{T}{1,5} \\ &= I \times (9,9 + 1,25) \times \frac{T}{1,5} \\ &= \frac{11,15}{1,5} \cdot I \cdot T = 7,433 \cdot I \cdot T \end{aligned}$$

Paso 3: Speedup de la parte de cálculo

$$S_{\text{cálculo}} = \frac{T_{\text{orig}}}{T_{\text{new}}} = \frac{10}{7,433} \approx 1,345$$

Paso 4: Speedup global con Ley de Amdahl

La fracción mejorada es $F_m = 0,9$ (el 90% es cálculo) con aceleración $A_m = 1,345$:

$$S_t = \frac{1}{(1 - 0,9) + \frac{0,9}{1,345}} = \frac{1}{0,1 + 0,669} = \frac{1}{0,769} \approx 1,3$$

Interpretación: aunque el nuevo procesador es 1,345 veces más rápido para el cálculo, la Ley de Amdahl limita el speedup global a 1,3 porque el 10% de E/S no mejora. Esto ilustra perfectamente el principio de Amdahl.

Ejercicio resuelto 3 — Comparación de dos mejoras de diseño con CPI

Enunciado. Se ejecutan operaciones FP con frecuencia 25%; CPI medio FP = 4. CPI medio otras instrucciones = 1,33. La instrucción FSQRT se ejecuta con frecuencia 2% sobre las FP y tiene CPI = 20. Se comparan dos alternativas: opción 1 (bajar CPI de FSQRT a 2) y opción 2 (bajar CPI medio de todas las FP a 2,5).

CPI original:

$$\text{CPI}_{\text{orig}} = 0,25 \times 4 + 0,75 \times 1,33 = 1 + 0,998 \approx 2$$

Opción 1: mejorar solo FSQRT

FSQRT representa el $0,02 \times 0,25 = 0,005 = 0,5\%$ de todas las instrucciones. Su CPI pasa de 20 a 2. El resto de instrucciones FP (el $0,25 - 0,005 = 0,245$) mantienen CPI = 4.

$$\text{CPI}_{\text{op1}} = 0,245 \times 4 + 0,005 \times 2 + 0,75 \times 1,33 = 0,98 + 0,01 + 0,998 = 1,988 \approx 1,910$$

Opción 2: mejorar todas las FP

$$\text{CPI}_{\text{op2}} = 0,25 \times 2,5 + 0,75 \times 1,33 = 0,625 + 0,998 = 1,623$$

Comparación:

$$\text{Speedup opción 2} = \frac{\text{CPI}_{\text{orig}}}{\text{CPI}_{\text{op2}}} = \frac{2}{1,623} \approx 1,23$$

La opción 2 es claramente superior (menor CPI, mayor speedup). Mejorar la instrucción FSQRT en solitario tiene un impacto mínimo porque su frecuencia de uso es muy baja (0,5% del total de instrucciones). Esto es una manifestación directa de la Ley de Amdahl: mejorar una parte pequeña del programa tiene poco efecto global.

Ejercicio resuelto 4 — CPI con predictor de saltos y salto retardado

Enunciado. Un programa corre con $\text{CPI} = 1,6$. El 20% de instrucciones son saltos; tasa de acierto de predicción = 75%; coste de fallo = 6 ciclos; coste de acierto = 0 ciclos.

Parte a) Mejorar el predictor al 90% de aciertos.

El CPI se descompone en CPI ideal más los ciclos perdidos por saltos:

Contribución de los saltos al CPI:

$$\Delta\text{CPI}_{\text{saltos}} = 0,20 \times (1 - 0,75) \times 6 = 0,20 \times 0,25 \times 6 = 0,30$$

$$\text{CPI}_{\text{ideal}} = 1,6 - 0,30 = 1,30$$

Con tasa de acierto del 90%:

$$\Delta\text{CPI}'_{\text{saltos}} = 0,20 \times (1 - 0,90) \times 6 = 0,20 \times 0,10 \times 6 = 0,12$$

$$\text{CPI}' = 1,30 + 0,12 = 1,42$$

Parte b) Además, aplicar técnica de salto retardado (*delayed branch*).

El salto retardado ocupa el hueco después del salto con una instrucción útil (en lugar de dejarlo vacío). Si el compilador *siempre* encuentra una instrucción para rellenar el hueco, el número de ciclos perdidos por fallo de predicción se reduce en 1 (se aprovecha 1 ciclo del coste de fallo):

Nuevo coste de fallo = 6 - 1 = 5 ciclos.

$$\Delta\text{CPI}''_{\text{saltos}} = 0,20 \times (1 - 0,90) \times 5 = 0,20 \times 0,10 \times 5 = 0,10$$

$$\text{CPI}'' = 1,30 + 0,10 = 1,40$$

Ejercicio resuelto 5 — MIPS vs. tiempo de ejecución: peligro de las métricas derivadas

Enunciado. Dos computadores A y B ejecutan el mismo programa. Datos: computador A: $NI = 1000 \times 10^6$ instrucciones, $\text{CPI} = 1$, $F = 4$ GHz. Computador B: $NI = 800 \times 10^6$, $\text{CPI} = 1$, $F = 4$ GHz.

MIPS de cada computador:

$$\text{MIPS}_A = \frac{F}{\text{CPI} \times 10^6} = \frac{4 \times 10^9}{1 \times 10^6} = 4\,000 \quad \text{MIPS}_B = \frac{4 \times 10^9}{1 \times 10^6} = 4\,000$$

Ambos tienen los mismos MIPS. Pero los tiempos de ejecución son:

$$T_A = \frac{NI \times \text{CPI}}{F} = \frac{10^9 \times 1}{4 \times 10^9} = 250 \text{ ms} \quad T_B = \frac{800 \times 10^6 \times 1}{4 \times 10^9} = 200 \text{ ms}$$

Conclusión: aunque los MIPS son iguales, el computador B es más rápido (200 ms vs. 250 ms) porque ejecuta *menos instrucciones* para resolver el mismo problema (su ISA o compilador es más eficiente). Esto demuestra que los MIPS no son una buena métrica comparativa: no capturan la complejidad semántica de las instrucciones ni las diferencias entre ISAs.

1.19 Resumen y conclusiones del Tema 1

Los conceptos fundamentales del Tema 1 que el alumno debe dominar para el examen son los siguientes:

1. **La ecuación de rendimiento** $T_{\text{CPU}} = N \cdot \text{CPI} \cdot T_{\text{ciclo}}$ y la capacidad de usarla para comparar diseños alternativos.
2. **La Ley de Amdahl** y su aplicación para calcular el speedup global cuando solo se mejora una fracción del programa.
3. **La potencia dinámica** $P = \frac{1}{2}CV^2f$ y el efecto de escalar voltaje y frecuencia sobre el consumo energético.
4. **La Taxonomía de Flynn** (SISD, SIMD, MISD, MIMD) y los subtipos de MIMD (UMA, NUMA, multicore).
5. **Las limitaciones de los MIPS y GFLOPS** como métricas comparativas, y la superioridad del tiempo de ejecución como métrica universal.
6. **Los benchmarks SPEC** y la normalización mediante media geométrica.
7. **Las técnicas de eficiencia energética:** clock gating, DVFS, power gating y overclocking (Turbo Boost).
8. **El agotamiento del modelo mononúcleo** y los motivos que llevaron a la transición al multinúcleo.

Recordatorio de fórmulas para el examen

$$T_{\text{CPU}} = N \cdot \text{CPI} \cdot T = \frac{N \cdot \text{CPI}}{F} \quad \text{MIPS} = \frac{F}{\text{CPI} \cdot 10^6}$$
$$\text{Speedup} = \frac{T_{\text{orig}}}{T_{\text{new}}} \quad S_t^{\text{Amdahl}} = \frac{1}{(1 - F_m) + F_m/A_m}$$

$$P_{\text{din}} = \frac{1}{2} CV^2 f \quad \text{Velocidad SPEC} = \sqrt[n]{\prod_{i=1}^n \text{SPEC}_i}$$

$$K_f = 1,45 K_v - 0,40 \quad K_d = K_v^2 \cdot K_f \quad \text{BWM} = \frac{N \cdot F}{IP \cdot \text{CPI}_{\text{core}}}$$